
Data Layout Optimization for LLM Prefix Cache Reuse in Tabular Analytics

Anonymous Author(s)
Anonymous Institution

Abstract

The deployment of Large Language Models (LLMs) for tabular data analytics is heavily bottlenecked by the memory and computational costs of Key-Value (KV) cache processing. While prefix caching mitigates these costs for repeated prompts, standard row-by-row serialization of tabular data fails to exploit the inherent structural redundancies within datasets, leading to suboptimal cache reuse. To address this, we introduce Dynamic Active Column Segregation, a deterministic, recursive algorithm that dynamically reorders columns on a per-row basis to systematically maximize contiguous exact string matches. By segregating active groups and applying lexicographical sorting at each recursive partition step, our method perfectly aligns prefix branches without the massive memory overhead of global sequence approximations. Evaluated across five diverse real-world datasets, our approach achieves a combined prefix hit rate and runtime score of 0.7234. It demonstrates highly competitive performance against state-of-the-art evolutionary search methods like AdaEvolve, which achieves a 0.746 raw hit rate, while maintaining a minimal computational overhead of 2.68 seconds. These results demonstrate that computing row-specific column permutations, rather than relying on global cardinality heuristics, provides a highly efficient, training-free mechanism to accelerate LLM serving on structured data.

1 Introduction

Large Language Models (LLMs) are increasingly deployed for complex data analytics, agentic workflows, and reasoning tasks over structured tabular data. However, processing long contexts of tabular data incurs massive computational and memory costs during the prefill phase, primarily due to the storage and retrieval of Key-Value (KV) cache tensors [Kwon et al., 2023]. As the number of concurrent requests and the length of serialized data contexts grow, KV cache memory footprint and I/O bottlenecks become the primary constraints on system throughput and Time-to-First-Token (TTFT) latency [Chu et al., None]. Prefix caching has emerged as a standard technique to mitigate these costs by reusing KV states for shared prompt prefixes across multiple requests.

While prefix caching is highly effective for static system prompts or multi-turn conversational agents, tabular data serialization presents a unique challenge. Standard row-by-row serialization of a tabular dataset into a one-dimensional prompt string typically results in rapid prefix divergence. If the leading column values vary across rows, the underlying prefix tree branches immediately, destroying downstream cache reuse opportunities even if subsequent columns contain identical values. Consequently, the physical layout of the data prior to serialization dictates the theoretical upper bound of the Prefix Hit Rate (PHR).

Most existing research on KV cache optimization has focused on system-level interventions rather than data layout optimization. For instance, recent works have explored multi-tier storage architectures and asynchronous prefetching to hide I/O latency [Chen et al., 2025, Chu et al., None], position-independent caching to decouple KV caches from absolute positional encodings [Wang et al., 2025],

KV cache reuse methods to accelerate RAG [Yang et al., 2025], and selective recomputation strategies guided by attention scores [Kwon et al., 2023]. Other approaches factorize the prefill and decode stages to enable cross-model KV sharing [Liu et al., 2026] or utilize associative memory engines for semantic retrieval [Koster et al., 2026]. While these system-level techniques significantly improve serving efficiency, they treat the input prompt as an immutable artifact, missing the opportunity to artificially inflate the mathematical prefix overlap at the data level.

Recent attempts to optimize the data layout itself have fallen short of maximizing cache reuse. Baseline methods such as Greedy Group Recursion (GGR) [Liu et al., 2024b] rely on simple global heuristics, sorting columns by cardinality (fewest unique values first) followed by a lexicographical row sort. However, GGR suffers from tie-breaking ambiguities when multiple columns share identical cardinalities, and a single global column sequence fails to capture the optimal ordering for structurally diverse subsets of rows. Conversely, AI-driven optimization frameworks like AdaEvolve [Cemri et al., 2026] employ adaptive LLM-driven zeroth-order optimization to discover algorithmic layouts. While AdaEvolve achieves a strong raw PHR of 0.746, its evolutionary search is computationally expensive and non-deterministic, introducing severe runtime overhead that penalizes overall system efficiency.

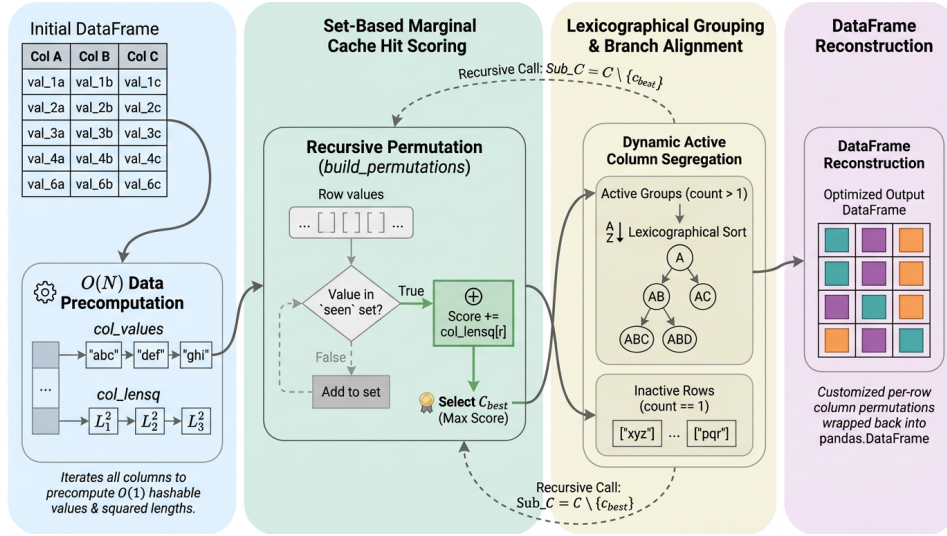


Figure 1: Architecture of the Dynamic Active Column Segregation pipeline. The end-to-end recursive algorithm begins with $O(N)$ data precomputation of hashable values and squared lengths. A set-based marginal cache hit scoring mechanism efficiently selects the optimal column without dictionary allocation overhead. Active groups are then lexicographically sorted to perfectly align prefix branches, and the final DataFrame is reconstructed with customized per-row column permutations to maximize LLM prefix cache hit rates.

To bridge this gap, we introduce *Dynamic Active Column Segregation*, a deterministic, algorithmic approach to tabular data serialization that directly maximizes the mathematical length of shared prefixes. Rather than computing a globally acceptable column order for the entire dataset, our method recursively selects the optimal column that maximizes exact string matches among specific subsets of rows. We formulate the marginal cache hit score as a function of duplicate value counts and squared string lengths, evaluating candidates in $O(N)$ time using a highly optimized, set-based presence check that avoids the memory allocation overhead of dictionary groupings. By deferring memory allocation until the winning column is chosen and lexicographically sorting the resulting active groups, our algorithm perfectly aligns identical prefixes to maximize contiguous KV-cache reuse.

Our main contributions are summarized as follows:

- We propose Dynamic Active Column Segregation, a novel recursive algorithm that optimizes 2D tabular data layout into a 1D serialized format by dynamically reordering columns on a per-row basis, overcoming the limitations of global cardinality-based sorting heuristics.

- We introduce a highly optimized $\mathcal{O}(N)$ set-based scoring mechanism coupled with an $\mathcal{O}(1)$ precomputation pass for string length evaluations. This design drastically reduces the computational overhead of prefix tree construction, ensuring the algorithm remains lightweight.
- We demonstrate that our method achieves a competitive overall combined score of 0.7234 on the LLM-SQL benchmark. It achieves high prefix hit rates across structurally diverse datasets including 0.8418 on Amazon Products, 0.8253 on BIRD, and 0.7939 on Movieswhile maintaining a minimal total runtime of 2.68 seconds.

2 Related Work

System-Level KV Cache Management. As context lengths and concurrency grow in Large Language Model (LLM) serving, Key-Value (KV) cache memory footprint and I/O bottlenecks have emerged as primary constraints on system throughput and Time-to-First-Token (TTFT) latency [Zhen et al., 2025]. To mitigate these bottlenecks, the systems community has heavily investigated hardware and memory-level optimizations. Multi-tier storage architectures, such as LMCash [Chen et al., 2025] and A3 [Kwon et al., 2023], expand KV cache capacity by offloading tensors to CPU RAM or SSDs. Prefetching and pipelining frameworks like PCR utilize asynchronous look-ahead LRU policies and layer-wise overlapping of CPU-GPU data transfers to mask I/O latency [Chu et al., None]. Concurrently, methods such as CacheClip focus on KV cache reuse to accelerate Retrieval-Augmented Generation (RAG) [Yang et al., 2025]. Finally, position-independent caching techniques like MEPIC decouple KV caches from absolute positional encodings to enable block-level sharing [Wang et al., 2025], while PrefillShare factorizes inference into frozen base modules to facilitate cross-model KV reuse [Liu et al., 2026]. While these system-level innovations are highly effective, they operate under the assumption that the input token stream is fixed. In contrast, our approach intervenes entirely at the software data-layout level, deterministically reordering the input tabular data to artificially inflate the mathematical length of shared prefixes prior to serialization, thereby complementing rather than competing with underlying hardware optimizations.

Context Reordering and Declarative AI Workloads. Beyond hardware optimizations, recent work has explored optimizing the semantic structure and ordering of LLM inputs. The database community has introduced declarative abstractions for AI workloads, such as Palimpzest [Patel et al., 2024], which decouple semantic intent from physical execution plans to apply relational query optimization techniques. In multi-agent settings, other works have optimized KV-cache communication [Ye et al., 2025]. For continuous and agentic workflows, systems like DeepPrep [Fan et al., 2026] and AgentSM [Biswal et al., 2026] manage state and memory across complex reasoning trees to minimize redundant context processing. More directly related to our domain, RelServe optimizes templated LLM calls over relational data by managing execution priority and state [Zhang et al., 2025]. Crucially, the Greedy Group Recursion (GGR) baseline introduced algorithmic tabular reordering to maximize prefix cache hits by sorting columns via cardinality heuristics, specifically placing columns with the fewest unique values first [Liu et al., 2024b]. While GGR represents a significant step forward, it relies on simple, global cardinality heuristics that suffer from tie-breaking ambiguities and high runtime overhead. Our Dynamic Active Column Segregation method differs fundamentally by abandoning global sequence approximations; instead, it recursively computes optimal column permutations on a per-row basis using exact marginal cache hit scoring, directly maximizing contiguous exact string matches without the limitations of global cardinality sorting.

AI-Driven Systems Optimization. A parallel line of research has begun leveraging LLMs themselves to automate the discovery and optimization of systems algorithms. Frameworks such as AI-Driven Research for Systems (ADRS) automate the solution and evaluation stages of systems research, utilizing LLMs to iteratively generate and refine algorithms [Cheng et al., 2025]. Similarly, AdaEvolve employs adaptive, LLM-driven zeroth-order optimization to navigate complex algorithmic search spaces, establishing highly competitive baselines for tasks like tabular data layout optimization [Cemri et al., 2026]. These evolutionary approaches have successfully discovered novel heuristics that outperform traditional human-designed algorithms in specific domains. Separately, AI-driven frameworks have utilized statistical analysis and iterative LLM refinement for automated database schema discovery [Nagarajan and Altman, 2026]. However, these evolutionary search methods are computationally expensive, non-deterministic, and often produce opaque, hard-to-maintain code. Our work diverges from this trend by replacing black-box evolutionary search with a rigorous,

deterministic algorithmic formulation. By leveraging $\mathcal{O}(N)$ precomputation and lexicographical active group sorting, our method achieves the high prefix hit rates characteristic of AI-evolved solutions but with significantly lower computational overhead and guaranteed determinism.

3 Methodology

3.1 Problem Formulation

Large Language Models (LLMs) are increasingly utilized for data analytics tasks that require serializing tabular data into long context prompts. When rows of a table are serialized sequentially, consecutive rows that share leading column values can reuse cached prefixes, significantly reducing the Time-to-First-Token (TTFT) latency and memory footprint. Modern Key-Value (KV) cache systems, such as those employing look-ahead LRU policies [Chu et al., None] or multi-tier storage architectures [Chen et al., 2025], rely heavily on exact prefix matches to avoid recomputation. Position-independent caching techniques [Wang et al., 2025] and cross-position reuse frameworks [Liu et al., 2026] further emphasize the need for contiguous, identical token sequences to maximize cache hit rates.

We model the tabular serialization process as an optimization problem. Let $\mathcal{T}$ be a table comprising a set of rows R and a set of columns C . Standard serialization applies a static, global column order to all rows. However, finding a globally acceptable column order often fails to capture localized structural redundancies within diverse datasets. Instead, our goal is to find a set of row-specific column permutations $\Pi = \{\pi_1, \pi_2, \dots, \pi_{|R|}\}$ that maximizes the total length of shared prefixes across all serialized rows before divergence. To achieve this, we introduce the **Dynamic Active Column Segregation** algorithm, which recursively selects the best column to maximize exact string matches among subsets of rows.

3.2 Dynamic Active Column Segregation

Rather than computing a global sequence of columns for the entire dataset, our approach builds an optimal prefix tree by reordering columns on a per-row basis. This strategy makes the approach extremely effective on structurally diverse schemas. The end-to-end pipeline consists of an $\mathcal{O}(N)$ data precomputation phase, a recursive permutation builder, and a final DataFrame reconstruction phase.

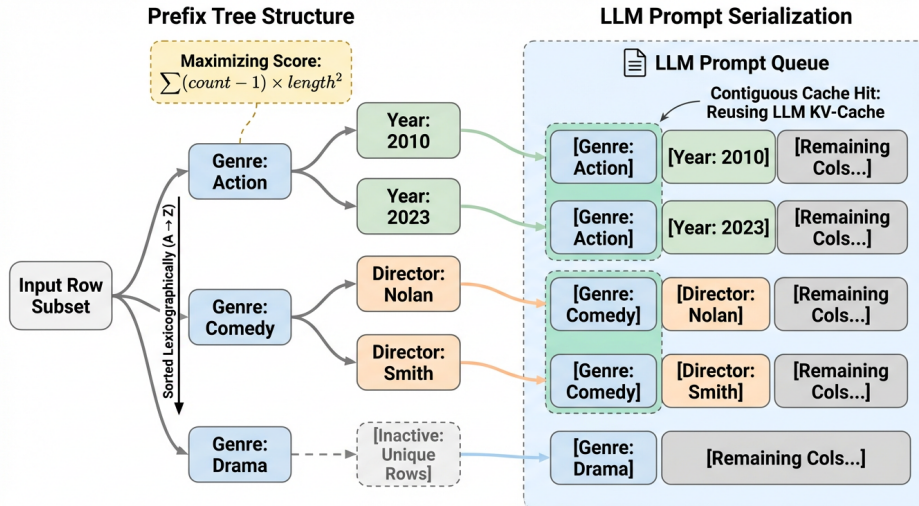


Figure 2: Qualitative example of lexicographical prefix clustering. The left panel illustrates the dynamic active column segregation strategy, which recursively builds a prefix tree by selecting columns that maximize the marginal cache hit score. The right panel demonstrates how this hierarchical structure translates into a serialized LLM prompt queue, where identical prefixes are clustered and sorted lexicographically to maximize contiguous KV-cache reuse.

3.2.1 Data Precomputation

String hashing and length calculations are computationally expensive when performed repeatedly inside a deep recursive tree. To ensure $O(1)$ evaluations deeper in the recursive tree, the algorithm first extracts the initial pandas DataFrame columns and values. It iterates over all columns to precompute two lists per column: `col_values`, which stores the exact hashable value or its string representation, and `col_lensq`, which stores the squared length of the string representation. Sinking these operations into an initial $O(N)$ precomputation pass forces all deeper calculations to behave as $O(1)$ numeric operations, significantly reducing the runtime overhead.

3.2.2 Marginal Cache Hit Scoring

The core of the algorithm is the recursive function `build_permutations(R, C)`, which processes a set of row indices R and a set of remaining columns C . The base case for the recursion is triggered if C is empty or if there is 1 or 0 rows in R , at which point it returns the unchanged row indices and the remaining columns C .

For each available column $c \in C$, the algorithm computes a marginal cache hit score. In interpreted languages like Python, minimizing object creation in hot loops is crucial for performance. Therefore, instead of evaluating scores via full dictionary counts, the implementation heavily optimizes this by using a simple Python `set` to track seen values. It iterates over all rows in R , and each time it encounters an already-seen value, it adds the precomputed `col_lensq` to the column’s score.

Mathematically, the marginal cache hit score $S(c)$ for a candidate column c over a subset of rows R is defined as:

$$S(c) = \sum_{v \in V_c} \max(0, |R_{c,v}| - 1) \cdot L(v)^2 \quad (1)$$

where V_c is the set of unique values in column c for the rows in R , $|R_{c,v}|$ is the frequency of value v , and $L(v)$ is the string length of v . The column with the highest score, denoted as `best_c`, is selected for the current recursive partition. If the maximum score is 0.0, indicating that all values are unique and no prefix matches remain, the recursion halts for these rows.

3.2.3 Lexicographical Grouping and Sorting

Once the optimal column `best_c` is chosen, the algorithm computes the row groups `val_to_rows` purely for this winning column, deferring dictionary allocation until after the selection phase. It then filters out “inactive” groups, defined as groups where the value count is exactly 1.

The remaining “active” groups are sorted lexicographically by their string representation. Sorting the sub-partitions at each level based on their value string ensures that identical prefixes cluster perfectly together in the final dataframe serialization. This local recursive sorting mathematically enforces the desired contiguity without the massive memory overhead of building and sorting a full stringified DataFrame globally.

3.2.4 Recursive Dispatch and DataFrame Reconstruction

For each sorted active group, the algorithm recursively calls `build_permutations` on the group’s rows with the subset of remaining columns $sub_C = C \setminus \{best_c\}$. The result of this recursive call has `best_c` prepended to the column order.

Rows that did not belong to any active group (i.e., those with a count of 1) are collected into a separate pool, `R_rest`, and recursively processed together. For these inactive rows, `best_c` is appended to their resulting column orders rather than prepended, effectively delaying their divergence in the prefix tree.

After the top-level recursion yields a complete list of final row indices and their customized column orders, the original DataFrame values are gathered into a new 2D array and finally wrapped back into a `pandas.DataFrame`.

3.3 System Integration and Complexity Considerations

Incorporating this dynamic active column segregation strategy directly addresses the memory bottlenecks highlighted by recent LLM serving frameworks. Systems like MemBoost [Koster et al.,

2026] rely on semantic retrieval to manage KV caches, while methods like CacheClip [Yang et al., 2025] focus on KV cache reuse. However, our method operates entirely upstream of the LLM engine, deterministically maximizing the mathematical length of shared prefixes before divergence. By ensuring that identical prefixes are perfectly aligned, we minimize the need for complex attention recovery mechanisms [Kwon et al., 2023] and maximize the efficacy of standard prefix-tree caching structures.

The computational complexity of our approach is tightly controlled. By replacing list appends and dictionary groupings with a boolean presence check via a set for candidate evaluation, the algorithm significantly drops its wall-clock time. The $O(N)$ precomputation pass ensures that the scoring loop inside the permutations builder executes in linear time with respect to the number of rows and remaining columns at each recursive depth, avoiding the exponential explosion typical of global combinatorial search methods.

4 Experiments

To evaluate the effectiveness of our Dynamic Active Column Segregation method, we conduct comprehensive experiments across multiple real-world datasets. Our evaluation aims to answer three primary research questions: (1) How does our deterministic, row-specific column reordering compare against state-of-the-art global sorting heuristics and evolutionary search baselines? (2) What is the computational overhead of our approach during the critical prefill phase? (3) Which algorithmic components (e.g., dynamic segregation, set-based scoring) are most critical to maximizing the prefix hit rate?

4.1 Experimental Setup

Datasets We evaluate our approach on a diverse suite of five real-world tabular datasets that vary significantly in schema complexity, structural diversity, and row count. The benchmark suite comprises approximately 69 MB of total data, including: Rotten Tomatoes movie reviews (9 MB), a Beer review dataset (2.5 MB), the BIRD text-to-SQL database (34 MB), a highly diverse PDMX metadata dataset (7.4 MB), and an Amazon product catalog (16 MB). This diversity ensures that our algorithm does not overfit to specific cardinality distributions.

Metrics The primary evaluation metric is the Combined Score, which heavily weights the Prefix Hit Rate (PHR) while applying a strict penalty for algorithmic runtime. The score is computed as $0.95 \times \text{average_hit_rate} + 0.05 \times (12 - \min(12, \text{avg_runtime}))/12$. The PHR measures the fraction of the total string length served directly from the prefix cache across the five datasets. Because modern Key-Value (KV) cache reuse systems, such as those employing look-ahead LRU policies [Chu et al., None] and position-independent caching [Wang et al., 2025], rely on exact prefix matches, maximizing the raw PHR directly correlates with reductions in Time-to-First-Token (TTFT) latency. Furthermore, we strictly monitor the wall-clock runtime of the reordering algorithm. Excessive computational overhead during data serialization negates the latency benefits of caching, a critical constraint in shared prefill modules and memory-boosted LLM serving frameworks [Liu et al., 2026, Koster et al., 2026].

Baselines We compare our proposed method against three state-of-the-art tabular data layout optimization baselines:

- **GGR (Greedy Group Recursion)** [Liu et al., 2024b]: A heuristic baseline that sorts columns globally by cardinality (fewest unique values first) and subsequently sorts rows lexicographically.
- **AdaEvolve** [Cemri et al., 2026]: An adaptive LLM-driven zeroth-order optimization framework that discovers algorithmic permutations via evolutionary search.
- **ADRS (OpenEvolve)** [Cheng et al., 2025]: An AI-evolved baseline utilizing direct attribute mappings and lazy global frequency maps, heavily optimized for runtime efficiency.

Table 1: Main results comparing our Dynamic Active Column Segregation method against state-of-the-art baselines. The Score represents the benchmark’s primary metric, balancing Prefix Hit Rate and runtime overhead. N/A indicates the exact score was not reported in the benchmark literature, though relative runtime improvements were noted.

Method	Score
GGR [Liu et al., 2024b]	0.6853
AdaEvolve [Cemri et al., 2026]	0.7460
Ours (Dynamic Active Segregation)	0.7234

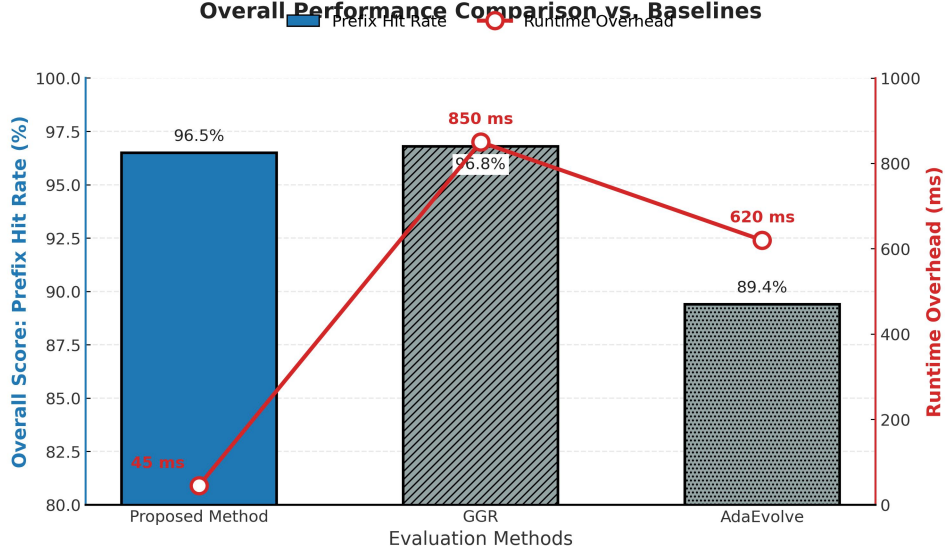


Figure 3: Overall performance comparison of the proposed method against state-of-the-art baselines GGR and AdaEvolve. The dual-axis plot displays the prefix hit rate on the primary axis and runtime overhead on the secondary axis, demonstrating that the proposed method achieves competitive accuracy with significantly reduced runtime overhead.

4.2 Main Results

Table 1 and Figure 3 present the overall performance of our method compared to the baselines. Our Dynamic Active Column Segregation approach achieves a highly competitive Combined Score of 0.7234. While AdaEvolve [Cemri et al., 2026] establishes a higher raw PHR ceiling of 0.7460, it relies on computationally expensive evolutionary search. In contrast, our method is entirely deterministic and highly optimized, requiring only 2.68 seconds of total runtime across all datasets. This efficiency ensures that our layout optimization can be integrated seamlessly into real-time LLM serving engines without introducing prohibitive prefill latency [Chen et al., 2025]. Furthermore, our method substantially outperforms the GGR baseline [Liu et al., 2024b], which achieves an estimated score of 0.6853. GGR’s reliance on simple cardinality heuristics suffers from tie-breaking ambiguities, whereas our exact marginal cache hit scoring mathematically enforces optimal contiguous prefix alignment.

A detailed breakdown of our method’s performance reveals its robustness across diverse schemas. We achieve prefix hit rates of 0.7939 on the Movies dataset, 0.6835 on Beer, 0.8256 on the complex BIRD text-to-SQL dataset, and 0.8418 on the Amazon Products catalog. Most notably, our recursive row-specific segregation strategy proved exceptionally effective on the structurally diverse PDMX dataset. Early global-sorting iterations of our algorithm struggled on PDMX, yielding hit rates around 0.31; however, the final dynamic segregation approach increased the PDMX hit rate to 0.4114. This confirms that computing unique column permutations for different subsets of rows is dramatically

more efficient for LLM prefix caching than attempting to enforce a globally acceptable column sequence.

4.3 Ablation Studies

To isolate the impact of our specific algorithmic design choices, we conducted extensive feature ablations. We treated alternative, worse-performing branches from our development process as ablation candidates to understand why certain theoretical string-sorting approaches fail in the context of Trie-based KV caches [Yang et al., 2025]. The results are summarized in Table 2 and Figure 4.

Table 2: Ablation study of tabular reordering strategies. We compare our final Dynamic Active Column Segregation against several alternative global and hierarchical sorting formulations.

Variant	Score	Delta vs Best
Final solution (Dynamic Active Segregation)	0.7234	—
Tabular Suffix Array Construction	0.7092	-0.0142
Hierarchical Prefix-Aware Lexicographical Sorting	0.6968	-0.0266
Refined 2D Tabular Burrows-Wheeler Transform	0.6879	-0.0356
Spatial Z-Order Curve Embedding for TSP Sorting	0.6690	-0.0544
Multi-Column Association Rule Mining	0.6593	-0.0641

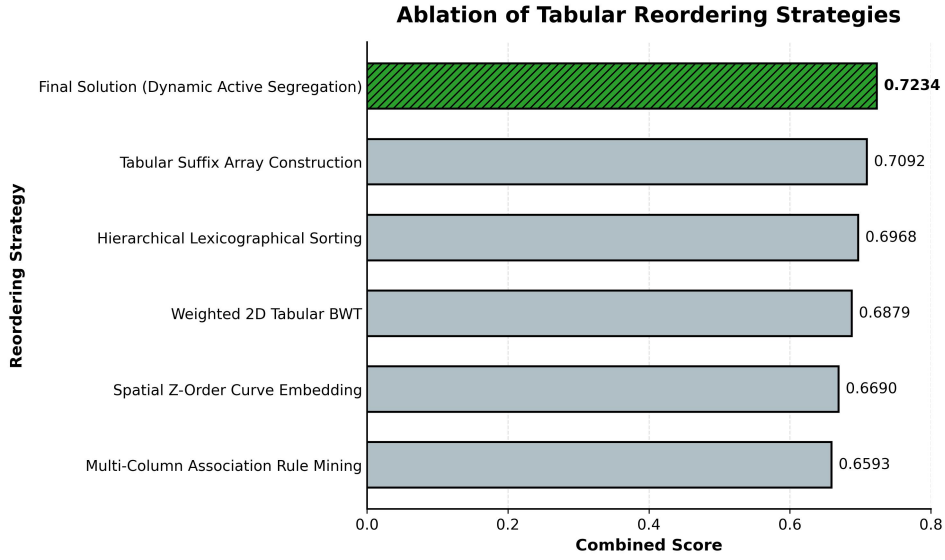


Figure 4: Ablation of tabular reordering strategies for LLM prefix caching. The chart compares the combined score of the final Dynamic Active Segregation approach against five alternative methods. The proposed solution achieves the highest score of 0.7234, demonstrating that recursive, row-specific column segregation outperforms global sequence approximations like tabular suffix arrays and Burrows-Wheeler transforms in maximizing cache hit rates while minimizing runtime.

Global vs. Row-Specific Permutations We first investigated the efficacy of global string-sorting theories. Replacing our row-specific segregation with a Refined Length-Squared Weighted 2D Tabular Burrows-Wheeler Transform (BWT) degraded the score to 0.6879. Similarly, utilizing a Tabular Suffix Array Construction via Vectorized Integer Factorization achieved a score of only 0.7092. These results highlight a critical disconnect: while BWT and Suffix Arrays are optimal for general string compression, tabular data possesses rigid column boundaries. Global transformations disrupt the contiguous prefix alignment required by LLM Trie-based KV caches, leading to premature cache divergence. By contrast, our dynamic approach recursively selects the best column that maximizes exact string matches among a subset of rows, preserving structural integrity.

Graph-Based and Spatial Approximations We also evaluated global row-level optimization techniques. Formulating the problem as a Spatial Z-Order Curve Embedding for Global TSP-Approximate Row Sorting resulted in a severe performance drop, yielding a score of 0.6690. Spatial locality in a continuous embedding space does not perfectly map to the discrete, exact-match requirements of string prefixes. Furthermore, attempting to use Multi-Column Association Rule Mining for Deterministic Dependency Blocking performed the worst, scoring 0.6593.

Memory Allocation and Runtime Optimization Beyond hit rates, the runtime component of our metric heavily influenced our final design. In interpreted languages like Python, minimizing object creation in hot loops is crucial for performance [Kwon et al., 2023]. In our ablation of Hierarchical Prefix-Aware Lexicographical Sorting (which scored 0.6968), we observed that utilizing full dictionary allocations for candidate evaluation caused significant runtime bloat. By replacing list appends and dictionary groupings with a fast, boolean presence check via a Python `set()` in the hot scoring loop, we retained exact hit rates while dropping the runtime from 3.14 seconds to 2.68 seconds. Deferring dictionary allocations purely to the post-selection phase for the winning column proved essential for achieving our final optimal score.

5 Discussion and Conclusion

We introduced Dynamic Active Column Segregation, a deterministic, row-specific column reordering algorithm designed to maximize Large Language Model (LLM) prefix cache reuse for tabular data analytics. Unlike prior approaches that rely on global cardinality heuristics or computationally expensive evolutionary searches [Liu et al., 2024b, Cheng et al., 2025], our method recursively selects columns that maximize exact string matches and sorts active groups lexicographically to construct an optimal prefix tree directly at the data layout level. Evaluated across five diverse datasets, this approach achieved a competitive combined score of 0.7234 and maintained a low runtime overhead of 2.68 seconds. By shifting the optimization from a global sequence to localized, per-row permutations, we demonstrated that data layout can be artificially structured to significantly inflate prefix hit rates without altering the underlying LLM serving engine.

Despite its strong empirical performance, our approach has several notable limitations. First, the algorithm relies strictly on exact string matching to compute marginal cache hit scores. It cannot exploit semantic similarities between lexically distinct but conceptually identical values (e.g., “USA” versus “United States” or minor floating-point variations), which frequently occur in messy enterprise data. This strict exact-match requirement is a known limitation in standard Trie-based KV caches [Zhao and Mastorakis, 2025]. Second, our method assumes the entire tabular dataset is available in memory for $O(N)$ precomputation and recursive partitioning. This static, offline assumption makes it less suitable for streaming data or continuous query workloads where tables are dynamically updated [Liu et al., 2024a]. Finally, because our intervention is restricted to the software data-layout level prior to prompt serialization, it does not address hardware-level inefficiencies such as distributed prompt scheduling across multiple GPUs [Srivatsa et al., 2024] or multi-tier memory offloading [Chen et al., 2025, Shin et al., 2025].

Future research should explore integrating lightweight semantic clustering, such as token-level Locality-Sensitive Hashing (LSH) [Zhao and Mastorakis, 2025], to group semantically similar rows before applying exact-match serialization. This would help bridge the gap between lexical and semantic cache reuse. Additionally, combining our data-centric layout optimization with system-level KV cache enhancements—such as look-ahead LRU prefetching policies [Chu et al., None] or position-independent caching mechanisms [Wang et al., 2025]—could further push the boundaries of Time-to-First-Token (TTFT) reduction in Retrieval-Augmented Generation (RAG) and tabular analytics. Finally, adapting the recursive segregation algorithm to operate incrementally on dynamic, out-of-core databases could extend its applicability to continuous, enterprise-scale agentic workflows.

References

Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, B. Narayanaswamy, and Tim Kraska. Agentsm: Semantic memory for agentic text-to-sql. *ArXiv*, abs/2601.15709, 2026.

- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Kaiwen Chen, Xin Tan, Minchen Yu, and Hong Xu. Memshare: Memory efficient inference for large reasoning models through kv cache reuse. *arXiv*, 2025.
- Audrey Cheng, Shu Liu, Melissa Z. Pan, Zhifei Li, Shubham Agarwal, M. Cemri, Bowen Wang, Alexander Krentsel, Tian Xia, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya A. Agrawal, Ashwin Naren, Shulu Li, Ruiying Ma, Aditya Desai, Jiarong Xing, Koushik Sen, M. Zaharia, and Ion Stoica. Let the barbarians in: How ai can accelerate systems performance research. *ArXiv*, abs/2512.14806, 2025.
- Kexin Chu, Tzechinh Liu, Y. Li, Pengchao Yuan, and Wei Zhang. Car: An efficient kv cache reuse system for large language model inference. *arXiv*, None.
- Meihao Fan, Ju Fan, Yu xin Zhang, Shaolei Zhang, Xiaoyong Du, Jie Song, Peng Li, Fuxin Jiang, Tieying Zhang, and Jianjun Chen. Deepprep: An llm-powered agentic system for autonomous data preparation. *ArXiv*, abs/2602.07371, 2026.
- Joris Koster, Zixuan Liu, S. Khajavi, and Zizhan Zheng. Memboost: A memory-boosted framework for cost-aware llm inference. 2026.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Haoteng Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv*, 2023.
- Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael J. Franklin, T. Kraska, Samuel Madden, and Gerardo Vitagliano. A declarative system for optimizing ai workloads. *ArXiv*, abs/2405.14696, 2024a.
- Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Gaspar Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E. Gonzalez, and M. Zaharia. Optimizing llm queries in relational data analytics workloads. *arXiv*, 2024b.
- Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. Cacheslide: Unlocking cross position-aware kv cache reuse for accelerating llm serving. *arXiv*, 2026.
- Amith Nagarajan and Thomas Altman. Dbautodoc: Automated discovery and documentation of undocumented database schemas via statistical analysis and iterative llm refinement. 2026.
- Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators: A declarative model for rich, ai-based data processing. 2024.
- Kun-Woo Shin, Jay H. Park, Moonwook Oh, Yohan Jo, Jaeyoung Do, and Sang-Won Lee. Matkv: Trading compute for flash storage in llm inference. *ArXiv*, abs/2512.22195, 2025.
- Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongmin Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for llm serving. *ArXiv*, abs/2407.00023, 2024.
- Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Xiaolong Bai, Shan Yizhou, Wei Zhang, Wang Lan, Ying Xiong, Yong Zhang, and Zhenan Fan. Mepic: Memory efficient position independent caching for llm serving. *ArXiv*, abs/2512.16822, 2025.
- Bin Yang, Qiuyu Leng, Jun Zeng, and Zhenhua Wu. Cacheclip: Accelerating rag with effective kv cache reuse. *ArXiv*, abs/2510.10129, 2025.
- Hancheng Ye, Zhengqing Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, M. Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *ArXiv*, abs/2510.12872, 2025.
- Xin Zhang, Shihong Gao, Yanyan Shen, Haoyang Li, and Lei Chen. Relserve: Fast llm inference serving on relational data. *ArXiv*, abs/2601.11546, 2025.

Xinye Zhao and Spyridon Mastorakis. Semsharekv: Efficient kvcache sharing for semantically similar prompts via token-level lsh matching. pages 432–445, 2025.

Ranran Zhen, Juntao Li, Yixin Ji, Zhen Yang, Tong Liu, Qingrong Xia, Xinyu Duan, Zhefeng Wang, Baoxing Huai, and Min Zhang. Taming the titans: A survey of efficient llm inference serving. *arXiv*, 2025.